
What the Final Patch Hides: Event-Level Regression Measurement for Coding-Agent Harnesses

Anonymous Author(s)

Affiliation

Address

email

Abstract

Coding agents are graded on the final patch alone, so nobody knows what they break in between. We commit the working tree after every mutating tool call and replay the benchmark’s previously-passing tests at every commit inside its pinned container, then apply this to mini-swe-agent 2.4.6 unmodified: its own loop, prompts, model layer, budgets and submission protocol, with only its environment object replaced.

Agents break existing tests often and almost always repair them. Across 80 runs on 40 SWE-bench Verified tasks, resolving 34 and 33 of 40 under two model families, 11 runs broke 135 previously-passing test functions, 100 of them distinct. The official grader, reading only the final tree, saw one. Neither did the agent: over the window each test was broken, no test command ran it in 6 of the 10 classifiable cases, so a method reading only the agent’s logs is blind to most of them. The undercount holds in our own harness and on SWE-bench Live.

The instrument is also a control. Replacing our harness’s model-written check with one built on those same tests takes resolve from 13 to 26 of 40 with no contaminated final tree, at 16 to 32 seconds of added wall-clock, though it is exploratory and circular by construction. Two things bound all of this: rollback’s clean trees are partly empty ones, since 17 of 37 runs ship a tree they never changed, and nine of the ten trees that killed the suite opened no timeline event at all. We will release the instrument, its validation protocol, and the archived timelines. Appendix A catalogues 28 measurement defects met while building it, 23 of which produced plausible numbers rather than errors.

1 Introduction

Benchmarks for coding agents grade the final patch. SWE-bench [1] and its Verified subset [2] apply the submitted patch and report whether the target tests pass and the previously passing ones still pass, and work on agent-caused regressions counts what the submitted patch breaks [8]. That convention cannot see what happens while the agent works, because an agent that breaks a test at step three and repairs it at step five submits a clean patch. Two questions are left open: how often do agents break existing behaviour during a run, and how much survives?

The answer is not what we expected. Agents break the oracle often and almost always repair it, so what the endpoint misses is transient. The agent’s own tests and its action log miss it too. We measure the timeline instead of the endpoint. Every mutating tool call becomes a commit on a git reference the agent cannot see, and the oracle is re-run at every commit inside the benchmark’s pinned container. An agent operating system needs this observation primitive before it can build a recovery primitive. We contribute:

1. **An instrument for event-level regression measurement**, applicable to an agent we did not write. Every mutating tool call becomes a commit on a hidden git reference, and the benchmark’s previously-passing oracle is replayed at every recorded state inside its pinned container (Section 3).
2. **The measurement on an unmodified public scaffold**. mini-swe-agent 2.4.6 keeps its own loop, prompts, model layer, budgets, and submission protocol. Only its environment object is ours. It resolves 34 and 33 of 40 under two model families and breaks 135 oracle test functions counted per run (100 distinct) across 11 of the 80 runs, one still failing at grade time. In 6 of the 10 classifiable cases its own test commands never ran the test it broke (Section 5.1).
3. **The same measurement across recovery policies**. The undercount holds in our harness and on SWE-bench Live. But nearly half the rollback arm’s runs ship a tree they never changed, so a clean final tree partly means no edits at all (Sections 5.2, 5.3, 5.5).
4. **Recovery policy depends on the monitor’s precision**. With a model-written check, rollback buys 4 fewer contaminated trees for 11 fewer resolved instances (1 against 5; 13 against 24). Replacing that check with the repository’s own oracle gets both: 26 resolved, no contaminated tree, and a lower cost per resolved instance than either. It is exploratory and circular by construction (Section 5.4).
5. **A catalogue of 28 measurement defects** found while building the instrument (Appendix A); 23 produced a plausible number rather than an error.

All measurements are exploratory, on development slices excluded from future confirmatory study.

2 Background and related work

Benchmarks and their validity. SWE-bench [1] grades a patch by tests that must go from failing to passing and tests that must keep passing. Verified [2] is the human-screened subset, deprecated as a capability metric [45]. Concerns about it have accumulated. They include contamination and memorisation [6], plausible-but-wrong patches overstating resolve by 6.2 points [36], and insufficient tests, which UTBoost [7] augmented to expose mislabelled patches affecting 24.4% of leaderboard entries. Rolling [3, 5], private [4], and long-horizon [23] alternatives came in response. TDAD [8] measured regressions in *submitted* patches on Verified (6.1% of previously-passing tests, 30.2% of non-empty patches) and reduced them with an impact-analysis map. Process-level benchmarks score intermediate states from logs [34, 35], and AgentLens [37] names regression cycles in 10.7% of passing OpenHands runs, against our 11 of 80 runs, or 29% of the 38 that made more than two tree-changing observations. A log-derived rate is bounded below by what the agent chose to run, and in 6 of our 10 classifiable cases it ran nothing that would have executed the broken test. We execute the oracle at every mutating call instead.

Agent scaffolds and recovery. SWE-agent [9], OpenHands [10], Agentless [11], and mini-swe-agent [12] span the scaffold design space [13, 14]. Within-episode recovery appears as self-reflection [15, 16], progress-gated recovery [17], checkpoint repair [18], and aligned checkpoints [40]. Provenance-based recovery is surveyed in [19]. Closest to us, Kim et al. [38] match logged edits against the gold diff and recover agents that reach a gold-identical patch and then destroy it. Gao et al. [39] bind verifier evidence to code states. Neither counts regressions or compares policies. Running the repository’s tests during a run is deployed practice [42, 43]. Operating-system framings [20] and the branch-context primitive [41] supply scheduling and fork, commit, and abort, but do not observe the tree between actions. The regression problem is program repair’s plausible-versus-correct patch problem [26, 28, 29], where regression tests are the main defence against overfitting [27]. Trajectory-level evaluation [21, 22] argues resolve rate alone is uninformative, diagnosing structure from recorded actions. An action log cannot show a test the agent never ran. Public leaderboard trajectories record actions but not the tree at each step, which the instrument commits.

3 Instrument

Figure 1 shows the harness and the instrument. **Observational timeline.** After every mutating tool call the working tree is committed to a git reference outside the agent’s view (a private index keeps

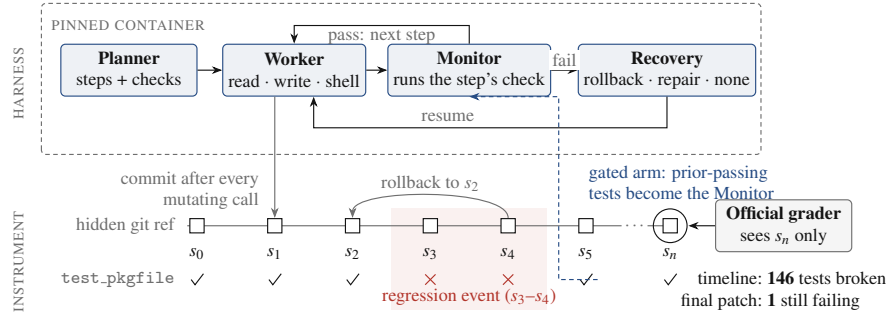


Figure 1: The harness and the instrument. The test row shows a previously-passing test regressing at s_3 – s_4 and repaired by a rollback to s_2 , which the official grader, reading only s_n , never sees. The counts are the GPT-5.6 rollback arm’s, over all 47 of its runs.

its own `git status` unchanged). Rollbacks and the run’s end are observations too, and an archived run can be re-measured without re-running the agent. The agent’s tools and the harness’s checks execute inside the same pinned container as the replay, synchronised to the host tree the timeline records (Appendix A.2).

Exhaustive replay. For each observation the instance’s previously-passing (PASS_TO_PASS) tests run inside the pinned container against that observation’s tree. A regression event is a test that passes at one observation and fails at a later one. We replay every observation instead of bisecting, because a recovery policy makes verdicts non-monotone. Replay is scoped to the files holding those tests, which keeps it affordable on Verified (4 to 8 seconds per observation, about 26 CPU-minutes per sweep). Live’s larger oracles cost tens of minutes per instance. Infrastructure failures during replay, and tests already failing in the unmodified image, are recorded as missing observations, not failures.

Validation. Five checks run before any paid experiment (negative and positive controls, the latter including *recovered* regressions, a flake screen, an unknown-rate ceiling, a baseline liveness check), plus a golden check driving the gold patch through the agent’s real tool path and a re-scoring control that re-measures an archived timeline with no model calls. Re-scoring four arms’ bearing runs reproduced their episode counts and probe sets exactly. Appendix C gives the detail.

4 Experimental setup

Benchmark and slice. SWE-bench Verified is a measurement substrate here, not a leaderboard. Its pinned images and oracles make the measurement possible. A 40-instance development slice (16 from `django`), stratified and fixed before any measurement, was used throughout and is excluded from future confirmatory study. The GPT-5.6 stack also ran its first 10 as a calibration, and plain rollback ran twice. SWE-bench Live [3], built monthly from live repositories, is the second substrate, graded by its own harness whose rules we mirror. Its slice was fixed before any outcome (Appendix B) and its oracles are about twenty times larger (median 1,189 against 58 tests).

Harness. A planner decomposes the task into steps with verification commands, a worker executes each with three tools (read, write, shell), and a monitor runs the verification. On failure the recovery policy acts: **rollback** (reset to the last verified checkpoint and retry), **repair-in-place** (retry from the failed tree), or **no recovery** (the FAIL is final and the run ends). Runs have a \$4 work-cost cap. One instance under one policy is a cell. Each (stack, policy, substrate) triple is an arm.

Public scaffold. `mini-swe-agent 2.4.6` [12] runs unmodified (its own loop, prompts, model layer, 250-step and \$3 caps, and submission protocol) with only its environment object replaced, so every bash command executes in the benchmark’s pinned container and the tree is committed to the hidden reference after each. The seam is inert but not invisible. The agent’s `git diff` and `git status` show its own uncommitted edits and none of the instrument’s commits, but the synthetic baseline carries a marker name 54 of the 80 runs saw, and those runs broke tests more often, though confounded with editing more (Appendix C). Appendix C lists six departures from its leaderboard configuration, and a grid census there puts its sampling inside our arms’ range: a median of 3 and 2 tree-changing observations per run against our 2 to 5, at 9 to 15 bash commands each. We claim no equivalence to

its published resolve rate, and because its budget differs in kind from our cap, no comparison between harnesses is tested.

Models. Two frontier stacks under identical harness, policy, instances and caps: `claude-opus-4-7/claude-sonnet-4-6` and `gpt-5.6-sol/gpt-5.6-terra` (planner/worker).

Outcomes. Resolve is the official grader’s verdict on the final patch. Events are reported at three levels because a few runs produce most: incidents (observations at which at least one oracle test broke), declared events (one per test function and onset, parametrised variants collapsed), and bearing runs (runs with at least one event). Incident *exposure*, a run’s incident count, is the pre-declared co-primary. An event and a final-state failure are different objects, so the undercount is in matched units (test functions broken during a run against those still failing at the end). Contamination counts oracle tests failing in the graded patch, net of baseline-dead. A tree that kills the suite is graded as failing every test but is a missing observation for event counting, so a contaminated cell need not be a bearing run.

Pre-declaration. The event unit, the proceed criteria, and the recovery-policy contrast were declared before the relevant data existed, and the analysis was committed before either comparison arm finished. One amendment preceded unblinding and one grading correction followed it (Section 5.3, Appendix B).

5 Results

5.1 An unmodified public scaffold breaks tests and puts them back

mini-swe-agent 2.4.6 ran unmodified on the same 40 instances. With `claude-sonnet-4-6`, 34 of 40 resolved (35 rot-aware) over 2,101 bash commands for \$31.38. Four runs hit its own \$3 cap. Three of those still grade resolved, because resolve reads the final tree, not the patch the scaffold never submitted. With `gpt-5.6-sol`, 33 of 40 resolved (34) over 937 commands for \$12.13. None of the 80 ended byte-identical to its start.

The timeline recorded 56 events in 5 of the 40 Claude runs and 79 in 6 of the 40 GPT-5.6 runs, on 8 instances, 3 bearing under both. Those 11 runs broke 135 oracle test functions counted per run (100 distinct, because the stacks overlap on the three shared instances). The official grader, reading only the final tree, caught exactly one of them: `test_pkgfile` in `pytest-6197` (Claude 0 of 56, GPT-5.6 1 of 79). The scaffold has no monitor or rollback, so every closure is an ordinary forward edit. The eleven stayed open for 1 to 22 bash commands (median 4), and three runs supply 81% of the 135. No run making two or fewer tree-changing observations (run and final excluded) was bearing, and 11 of the 38 that made more broke something (29%). Much of what the endpoint misses is a transient inside one logical edit, but transients are not free: 14% of what these runs cost (\$1.29 of \$9.52) was spent while a regression was open.

One run in full. On `django-12965` the GPT-5.6 scaffold read `SQLDeleteCompiler` for twelve commands, then at command 13 changed `single_alias` to count active aliases with `<= 1`. Its own reproduction script crashed, and at command 15 `runtests.py delete` printed `ERROR: test_fast_delete_large_batch`. Command 16 replaced the edit, command 18 ran the suite clean, and the run graded resolved, 52 of 52 previously-passing tests. The timeline holds all of this. The graded patch holds none of it.

Would its own testing have caught it? For each breaking edit we read the trajectory over the window the test was broken, from the breaking command to just before the repair, asking whether a test runner ran, whether any run would have executed *this* test, and whether its output showed the failure. Coverage is three-valued, with *unknown* for runs that stopped early or were truncated (Appendix C). In 6 of the 10 classifiable cases the agent’s own test commands never ran the broken test. In the other 4 they ran it and printed the failure by name, and it was repaired anyway. The eleventh is unknown, a `pytest -x` run that stopped earlier. Self-testing is blind to the first group and ignored on the second. The endpoint sees neither. With eleven cases this is a case series, not a rate.

5.2 The same undercount in our harness

Eight of the 47 GPT-5.6 rollback runs (the 37-run sweep and a 10-instance calibration) broke a previously-passing test, 146 test functions counting each run separately (145 distinct). One of those,

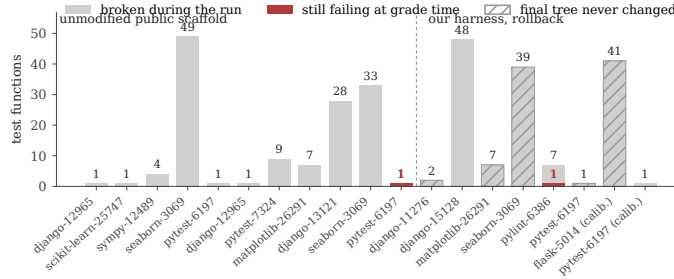


Figure 2: Every run that broke an oracle test, in matched units. Test functions broken on the timeline, dark where still failing at grade time. Left, the unmodified public scaffold under both models. Right, our rollback arm, hatched where the final tree is byte-identical to its start.

176 `test_can_read_toml_env_variable` in `pylint-6386`, was still failing at grade time. Three storms
 177 supply 88% of the 184 declared events over all 47 runs. Table 1’s 140 events and 104 test functions
 178 are the sweep alone, and declared events are exposure only.

179 **Attrition and rates.** Under rollback the Claude stack resolved 13 of 40 (32.5%, exact 95% CI 18.6%
 180 to 49.1%) for \$34.24 and the GPT-5.6 stack 13 of 40 for \$8.14. A circuit breaker stopped three cells
 181 and a synchronisation defect lost two, so 37 were attempted and 35 graded (13 of 35, 37.1%). Rates
 182 are out of 40 unless stated, and one instance is unresolvable under the current parser (Appendix A.3),
 183 putting the ceiling at 39 for every arm, the scaffold included.

184 A clean tree does not always represent work. Excluding the hidden reference, 17 of the 37 rollback
 185 runs end byte-identical to their start. Four of the six bearing sweep runs shipped nothing. Over the
 186 two that delivered a patch, 1 of 55 broken test functions is still failing, so the undercount rests on the
 187 public-scaffold arm, where no run ends unchanged.

188 **Model stack.** Event frequency also differed by model stack, but does not generalise. Under the same
 189 harness and policy the Claude stack produced no events in 40 runs against the GPT-5.6 stack’s 140 in
 190 6 of 37 (one-sided Fisher $p = 0.010$; reclassify three of the six as non-bearing and $p = 0.11$), yet the
 191 same `claude-sonnet-4-6` worker under unmodified `mini-swe-agent` is bearing on 5 of 40 with 56
 192 events, and the difference is absent on Live. Model and adapter are confounded, and controls are in
 193 Appendix C.

194 A break that stops the suite from being collected opens no timeline event, because a test the parser
 195 never mentions counts as a missing observation, not a failure, while official grading scores such a
 196 patch as failing everything. Nine of the ten suite-killed trees across all arms recorded no event, so the
 197 two instruments miss different things.

198 5.3 Recovery policy: what the clean final tree costs

199 On the primary endpoint, contamination paired by instance, no recovery was worse than rollback
 200 on 9 instances and better on 1 (exact sign test $p = 0.021$). Repair-in-place was worse on 5, better on
 201 1 ($p = 0.22$). Incident exposure, the co-primary, did not differ ($p = 0.45$ and 1.0). The arms cannot
 202 show the policy caused the difference. No recovery ends at the monitor’s first FAIL, so its regressions
 203 arrive on the last tree-changing observation with nothing after them, and its 19-of-19 persistence is
 204 the stopping rule as much as the policy.

205 Rollback pays for its clean tree in delivered work, resolving 13 of 40 against 24 and 22, losing nine
 206 discordant pairs to repair-in-place and winning one (McNemar $p = 0.021$). The loss is the monitor’s.
 207 Of the 18 failed rollback runs, 15 failed at the first step, and 9 of those instances were resolved by an
 208 arm that kept work the check rejected and the grader accepted.

209 **Disclosure.** The first unblinding gave $p = 0.375$, because seven cells whose final trees killed the suite
 210 had been dropped as ungradable. Correcting the grader and re-grading those archived trees gives the
 211 0.021 above, a rule change made after the data were seen (Appendix B).

Table 1: Every arm on both substrates. **contam.**: graded cells failing an oracle test, net of baseline-dead. **empty**: runs ending byte-identical to their start. **events**: declared (test function, onset) pairs, exposure only, never a denominator. **inc.**: incidents. **broken** $\rightarrow$ **left**: oracle tests broken during the bearing runs against those still failing at grade time. The Claude rollback arm’s 1 is baseline rot, not agent damage (Appendix C). The public-scaffold rows ran under mini-swe-agent’s own budget, not our cap. Composition and provenance: Appendix C.

substrate	arm	resolved	contam.	empty	events	inc.	bearing	broken $\rightarrow$ left	spend
Verified	mini-swe-agent, Claude	34	0	0/40	56	5	5/40	56 $\rightarrow$ 0	\$31.38
Verified	mini-swe-agent, GPT-5.6	33	1	0/40	79	6	6/40	79 $\rightarrow$ 1	\$12.13
Verified	GPT-5.6 rollback	13	1	17/37	140	11	6/37	104 $\rightarrow$ 1	\$8.14
Verified	Claude rollback	13	1	19/40	0	0	0/40	—	\$34.24
Verified	GPT-5.6 gated	26	0	4/40	70	7	5/40	62 $\rightarrow$ 0	\$10.51
Verified	GPT-5.6 split-gated	28	0	3/40	14	4	3/40	14 $\rightarrow$ 0	\$9.40
Verified	GPT-5.6 repair-in-place	24	5	0/40	84	7	7/40	83 $\rightarrow$ 4	\$14.73
Verified	GPT-5.6 no recovery	22	9	1/40	19	3	3/40	19 $\rightarrow$ 19	\$4.40
Live	GPT-5.6 rollback	0 (2)	0	25/39	156	9	6/39	147 $\rightarrow$ 0	\$10.29
Live	Claude rollback	2 (3)	1	24/39	49	6	3/39	48 $\rightarrow$ 0	\$49.80
Live	GPT-5.6 gated	1 (4)	2	9/39	280	6	3/39	270 $\rightarrow$ 0	\$11.49
Live	GPT-5.6 no recovery	4 (6)	4	3/39	30	4	4/39	30 $\rightarrow$ 30	\$8.95

212 5.4 Replacing the check removes the tradeoff

213 Replacing the planner-written check with the repository’s own oracle takes the GPT-5.6 stack’s
214 resolve from 13 of 40 to **26 of 40 (65.0%)** with **no contaminated final tree**, against the ungated arms’
215 24 and 22, and 4 empty patches of 40 against rollback’s 17 of 37. A step is rejected only if a test that
216 passed at the start now fails. This fourth arm was added after unblinding and is exploratory. The gate
217 runs at least 4 times in the median run at 4 to 8 seconds each, so 16 to 32 seconds of wall-clock and
218 \$2.37 more model spend than rollback, and per resolved instance it costs \$0.40 against rollback’s
219 \$0.63, repair-in-place’s \$0.61 and no recovery’s \$0.20 with nine contaminated trees. The split gate
220 leaves none at \$0.34. It beat rollback on 35 shared instances, winning 10 discordant pairs and losing
221 1 ($p = 0.012$), and left no contaminated tree against repair-in-place’s 5.

222 The gate gives a guarantee, not a rate, and it is circular. Its tests are the benchmark’s own oracle
223 (median ratio 1.00), so a clean tree and part of the resolve gain both follow whenever the gate saw the
224 final tree and the grader reads what the gate read. A fifth arm weakens that, reading a deterministic
225 half of the ids (2,278) and never the other half (2,585), on which the grader also rules [44]. The split
226 is over ids, not files, so a break reaching a held-out id usually reaches a watched one, and the held-out
227 half bounds the leak instead of sampling it. A file-level or non-oracle gate would settle it. It resolved
228 **28 of 40 (70.0%)**, indistinguishable from the full gate (5 discordant pairs to 3, $p = 0.73$) and better
229 than rollback (13 to 2, $p = 0.007$), with no held-out failure apart from the parser-capped instance.

230 5.5 Replication on SWE-bench Live

231 Live keeps the undercount and loses the resolve rates, at most 6 of 40 even rot-aware. Rollback broke
232 147 test functions across 6 of 39 runs, none still failing, against Verified’s 104 across 6 on oracles
233 twenty times larger. But 25 of the 39 shipped an unchanged tree, so the clean result is partly no
234 result. The gate contaminated two trees here, one of them after passing every check (Appendix C).
235 No recovery left all 30 of its own ($p = 0.125$).

236 6 Discussion

237 **Why timeline regressions matter.** A repaired regression still costs money. Work done while one
238 was open took 14% of the scaffold’s bearing-run cost and 27% of our harness’s (\$0.63 of \$2.33), both
239 small-base estimates. Patch verifiers [24] and process reward models [25] need step-level ground
240 truth, and instance-scale training sets [32, 33] do not carry it. The events supply it, and they tell a
241 recovery primitive [41] when to abort. Two rules would have caught Appendix A’s A and B classes,
242 14 of the 28: record infrastructure failures as missing observations, and require positive evidence that
243 the measurement path is live. Two changes follow: a recovery monitor should run tests rather than
244 trust a model-written check, and a benchmark should report what broke during the run beside the
245 final verdict.

246 **Limitations.** Every rate estimates a 40-instance slice with one sweep per arm, and event counts
247 bound the truth from both sides (Appendix C).

References

- [1] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR 2024. arXiv:2310.06770.
- [2] OpenAI. Introducing SWE-bench Verified. OpenAI blog, August 2024. openai.com/index/introducing-swe-bench-verified.
- [3] L. Zhang et al. SWE-bench Goes Live! NeurIPS 2025 Datasets and Benchmarks. arXiv:2505.23419.
- [4] X. Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? arXiv:2509.16941, 2025.
- [5] I. Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. NeurIPS 2025. arXiv:2505.20411.
- [6] S. Liang et al. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of reason. arXiv:2506.12286, 2025.
- [7] B. Yu et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. ACL 2025. arXiv:2506.09289.
- [8] P. Alonso, S. Yovine, and V. A. Braberman. TDAD: Test-driven agentic development: Reducing code regressions in AI coding agents via graph-based impact analysis. arXiv:2603.17973, 2026.
- [9] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024. arXiv:2405.15793.
- [10] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. ICLR 2025. arXiv:2407.16741.
- [11] C. S. Xia et al. Agentless: Demystifying LLM-based software engineering agents. FSE 2025. arXiv:2407.01489.
- [12] K. Lieret and C. E. Jimenez. mini-swe-agent. Software, 2025. github.com/SWE-agent/mini-swe-agent.
- [13] X. Wang et al. Executable code actions elicit better LLM agents. ICML 2024. arXiv:2402.01030.
- [14] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. ICLR 2023. arXiv:2210.03629.
- [15] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023. arXiv:2303.11366.
- [16] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023. arXiv:2303.17651.
- [17] A. Kadu and A. Krishnan. ReflexGrad: Within-episode failure recovery in LLM agents via progress-gated dual-process routing. arXiv:2511.14584, 2025.
- [18] P. Mazaheri. REPOT: Recoverable program-of-thought via checkpoint repair. arXiv:2605.30052, 2026.
- [19] Y. Wang et al. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. arXiv:2606.04990, 2026.
- [20] K. Mei et al. AIOS: LLM agent operating system. COLM 2025. arXiv:2403.16971.
- [21] S. Kapoor et al. Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. arXiv:2510.11977, 2025.
- [22] R. Shu et al. What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv:2607.06184, 2026.

291 [23] T. Le et al. SWE-EVO: Benchmarking coding agents in long-horizon software evolution
292 scenarios. arXiv:2512.18470, 2025.

293 [24] M. Raghavendra et al. Agentic rubrics as contextual verifiers for SWE agents. ACL 2026.
294 arXiv:2601.04171.

295 [25] M. L. Dihan and M. A. R. Khan. SWE-Shepherd: Advancing PRMs for reinforcing code agents.
296 arXiv:2604.10493, 2026.

297 [26] Z. Qi, F. Long, S. Achour, and M. Rinard. An analysis of patch plausibility and correctness for
298 generate-and-validate patch generation systems. ISSTA 2015. doi:10.1145/2771783.2771791.

299 [27] Y. Lou et al. When automated program repair meets regression testing: An extensive study on
300 two million patches. ACM TOSEM 33(7), 2024. arXiv:2105.07311.

301 [28] Z. Fei et al. Patch correctness assessment: A survey. ACM TOSEM 34(2), 2025.
302 doi:10.1145/3702972.

303 [29] H. Ye, M. Martinez, and M. Monperrus. Automated patch assessment for program repair at
304 scale. Empirical Software Engineering 26(2), 2021. doi:10.1007/s10664-020-09920-w.

305 [30] SWE-bench issue #601: Test result hijacking via stdout forging in evaluation harness.
306 github.com/SWE-bench/SWE-bench/issues/601, June 2026.

307 [31] OpenHands issues #4235 (October 2024) and #7044 (March 2025): Docker and environment
308 errors in SWE-bench instance evaluation. github.com/OpenHands/OpenHands.

309 [32] J. Yang et al. SWE-smith: Scaling data for software engineering agents. NeurIPS 2025 Datasets
310 and Benchmarks. arXiv:2504.21798.

311 [33] J. Pan et al. Training software engineering agents and verifiers with SWE-Gym. ICML 2025.
312 arXiv:2412.21139.

313 [34] M. B. Madiraju and M. S. P. Madiraju. RigorBench: Benchmarking engineering process
314 discipline in autonomous AI coding agents. arXiv:2606.22678, 2026.

315 [35] J. Chen et al. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous
316 integration. arXiv:2603.03823, 2026.

317 [36] Y. Wang, M. Pradel, and Z. Liu. Are "solved issues" in SWE-bench really solved correctly? An
318 empirical study. ICSE 2026. arXiv:2503.15223.

319 [37] P. Sahoo et al. AgentLens: Revealing the lucky pass problem in SWE-agent evaluation.
320 arXiv:2605.12925, 2026.

321 [38] M. Kim et al. Coherence collapse: Diagnosing why code agents fail after reaching the right
322 code. arXiv:2603.24631, 2026.

323 [39] X. Gao, J. Yang, and Q. Yang. Looping is not reliability: State-bound evidence and typed
324 revision contracts for agentic code repair. arXiv:2607.24604, 2026.

325 [40] Y. Zhuang et al. AgentRewind: Recoverable execution for long-horizon LLM agents.
326 arXiv:2608.14380, 2026.

327 [41] C. Wang and Y. Zheng. Fork, explore, commit: OS primitives for agentic exploration.
328 arXiv:2602.08199, 2026.

329 [42] Y. Chen, T. Ahmed, R. Jabbarvand, and M. Hirzel. Can old tests do new tricks for resolving
330 SWE issues? FSE 2026. arXiv:2510.18270.

331 [43] P. Gao et al. Trae Agent: An LLM-based agent for software engineering with test-time scaling.
332 arXiv:2507.23370, 2025.

333 [44] T. Ahmed, J. Ganhotra, A. Shinnar, and M. Hirzel. Investigating test overfitting on SWE-bench.
334 arXiv:2511.16858, 2025.

335 [45] OpenAI. Why SWE-bench Verified no longer measures frontier coding capabilities. OpenAI
336 blog, February 2026. openai.com/index/why-we-no-longer-evaluate-swe-bench-verified.

A The measurement defect catalogue

A.1 The catalogue by mechanism

Each row is a defect found while building the instrument. **S**: it produced a plausible number rather than an error. **G**: it was present while the test suite passed (A4 is the suite). **H**: it was findable only on a clean host. Class A rows depend on the machine the code runs on; class B rows render an infrastructure failure as a measurement; class C rows measure something other than the event as defined; class D rows consume a producer’s output without checking the producer.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherited the machine’s git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Validation probe invoked bare python	every probe exit 127 on a clean host	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare pytest from PATH	15 tests fail on a clean host as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare python	score 0.0 from a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the container	26 of 28 zero-step runs; see A.2	✓	✓	✓
B1	Output marker used shell :	a 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework’s results outside the parsed slice	✓	✓	
B3	Probe diffed against a deleted commit	every graded test a hole	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances fail far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127	✓	✓	
B6	Isolation removed loopback, not only the internet	23.5% of the oracle dead	✓	✓	
B7	Provider cache served removed containers	later cells replay against nothing and score clean	✓	✓	
B8	Tree reset reverted image build-time edits	one repository family’s oracle dead	✓	✓	
C1	Bisection assumed monotone verdicts	recovered regressions invisible	✓	✓	
C2	Declared event unit not implemented	rate off by up to 2.7×	✓	✓	
C3	Rollbacks produced no observation	recoveries invisible to the timeline	✓	✓	
C4	"Persists past the step boundary" undefined	one reading excludes the events of interest	✓	✓	
D1	Audit field never populated	0 tool errors reported, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost as task failures	✗	✓	
D3	Dependency install counted as agent work	attribution vacuous	✓	✓	
D4	Join took the first observation, not the last	failures pinned to the wrong tree	✓	✓	
D5	Executed config rebuilt from a name	manifest described a different run	✓	✓	
D6	Git handles never released	sweeps die after ~400 cells	✓	✓	
D7	Console re-walked the tree per run	presented as a dead server	✗	✓	
D8	Detection events lacked the ids attribution joined on	attributed detection always unknown	✓	✓	
D9	Absent coverage rendered as measured silence	unmeasured regressions reported as silent	✓	✓	
D10	Output-capped turns executed	half-written file; a 78-event storm	✓	✓	

Totals: 23 of 28 silent; 27 of 28 present under a passing suite; 6 of 28 findable only on a clean host. Ten further defects found after this census closed are described where they arose (Sections 5.2 and 5.3, Appendices A.3 and C); the rest will be released with the code.

A.2 The defect that invalidated a conclusion

After nineteen fixes, every validation check passed, the re-scoring control reproduced archived episode counts exactly, and three pilots (80 runs) measured an event rate far below the pre-declared proceed criterion. We drafted the conclusion the rule directed: the benchmark offered too little opportunity, so switch benchmarks. The conclusion was wrong. The harness ran the agent on the host in a bare source checkout, while every probe and every gate ran inside the pinned container. On the host, `import matplotlib` in that checkout succeeds by importing the uncompiled source tree as a namespace package, and everything downstream fails in ways indistinguishable from an incompetent agent. Twenty-six of 28 zero-step runs traced to this. The checks had validated the measurement path and never the agent’s execution path. The fix was to route the agent’s tools and checks through the same container as the replay, and to add a per-cell environment parity check that runs before any model call.

360 A.3 Mechanisms in public infrastructure

361 Class A: OpenHands issues #4235 and #7044 [31], environment construction failures against SWE-
362 bench images. Class B: UTBoost’s finding that the held-out oracle is insufficient at leaderboard scale
363 [7], and, on SWE-bench Live, baseline tests that fail in the unmodified image because the calendar
364 passed a deprecation date encoded in the instance. Class C: final-state regression measurement [8].
365 Class D: the SWE-bench grader accepting forged test output on stdout [30], and, at the harness’s
366 current revision, a parser that keys parametrised ids by their full text while the dataset stores them
367 truncated at the first space (the earlier parser’s behaviour), so 16 previously-passing ids of one instance
368 in our slice match no output and every submission to it grades as unresolved.

369 B Pre-declaration and analysis

370 The event unit is one (test function, onset observation) pair with parametrised variants collapsed. The
371 gates for proceeding on a substrate were an event rate of at least 0.30 per run and a bearing fraction
372 of at least 25%. Both stacks failed the conjunction (bearing 0% and 16.2%). We then re-declared the
373 gates per (substrate, model stack) before the contrast was unblinded, so no arm here is confirmatory.
374 The contrast’s primary endpoint is final-state contamination, paired by instance, exact two-sided
375 sign test. The co-primary is incident exposure. The analysis script was committed before either
376 comparison arm finished. The regression-gated arm was added after the contrast was unblinded and
377 is exploratory. No multiplicity adjustment was planned or applied. Budget exhaustion is scored as
378 failure. Instances whose baseline oracle records failures in the unmodified image are excluded from
379 resolve-rate comparability claims, and their dead tests are typed as missing observations for event
380 counting.

381 **Compute.** Every run, replay, and grade ran on one 12-vCPU, 22 GB x86 virtual machine running
382 Docker, with no GPUs. The six harness arms on Verified in Table 1 cost \$81.42 in model spend and
383 the two public-scaffold arms \$43.51, plus \$6.87 for the second rollback sweep. Calibration and pilots
384 cost a comparable amount, and the Live sweeps report their own spend in Table 1.

385 **Slice construction on SWE-bench Live.** Fixed before any outcome: pytest-parsed instances with
386 27 to 3,000 previously-passing tests, at most four per repository, earliest first, then the first 40 of 48
387 candidates whose image passed a model-free environment check. No coverage maps are built for
388 Live, so attribution there is unknown.

389 **Attribution (Verified only).** A detected regression is classified three ways: *attributed* if a failing
390 harness check and the broken test both exercise a file the agent changed at that observation (per-
391 instance coverage map at the base commit), *co-occurring* if some check failed while the regression
392 was open without such a link, and *unknown* if the map cannot say. Of the calibration’s 57 raw
393 episodes, 39 were attributed to a failing check, 16 co-occurred with one, and 2 were silent. Of the
394 sweep’s 162, 48 had none. The public-scaffold arm has no monitor, so attribution is undefined and no
395 silence statistic is reported.

396 **Grading correction between unblindings.** The first unblinding of the recovery-policy contrast gave
397 $p = 0.375$ for the primary endpoint, because seven of the eight cells whose final trees made the suite
398 uncollectable (five no-recovery, two repair-in-place) were dropped as ungradable. Official grading
399 scores such a patch as failing every test, so the most contaminated states had been dropped from the
400 endpoint they bore on most. The grader, not the replay, now distinguishes an environment failure
401 from a patch that kills the suite, and we re-graded the seven archived trees without re-running any
402 agent.

403 C The public-scaffold arm

404 **The seam.** mini-swe-agent’s loop touches the working tree in one place:
405 `DefaultAgent.execute_actions` calls `env.execute(action)`. We replace the environ-
406 ment object and nothing else, so the scaffold keeps its own loop, prompts, litellm model layer, step
407 and cost limits, and submission protocol. Our environment runs each bash command inside the
408 benchmark’s pinned container, pulls the resulting tree back to the host the timeline records, and
409 commits an observation when the tree changed. We measured two properties of the seam instead of
410 assuming them. First, the instrument does not advance the container’s git state, so the agent’s own

411 `git status`, `git log`, and `git diff` show its uncommitted work and none of the instrument's
412 commits. The first paid pilot ran before this was true, and it spent eight of twenty commands
413 discovering that its edits had been committed underneath it. Second, the instrument is inert but
414 visible. The synthetic baseline commit it creates carries a fixed marker name, and 54 of the 80 runs
415 saw that name in some command's output, 8 of them through `git status` (departure 2). We cannot
416 settle here whether seeing it changed behaviour. The 54 runs that saw it resolved 43 and were bearing
417 on 10, and the 26 that did not resolved 24 and were bearing on 1 (one-sided Fisher $p = 0.069$). A run
418 that edits more has both more to see and more to break, and that explains part of this, since no run
419 with two or fewer tree-changing observations was bearing at all under either condition. But within
420 the runs that edited more the association does not vanish (10 of 27 against 1 of 11, $p = 0.088$), so we
421 report it as an open question about the seam and not a settled confound. We also wrap the routed
422 command so that a command ending in a here-document terminator or a comment parses as the
423 scaffold's own `bash -c` would parse it. Before that fix, 39 of 40 GPT-5.6 cells hit a shell syntax
424 error, because that model writes every script as a here-document.

425 **Departures from the scaffold's leaderboard configuration.** (1) The measurement container has
426 no network, while its own deployment uses Docker's default bridge. The unchanged prompt tells
427 the agent it may install tools, and here it cannot. (2) The container's git history is a single synthetic
428 commit, because the image's upstream history can contain the fix the task asks for. (3) Resolve is
429 graded on the final tree by the official grader, not on the scaffold's submitted patch. (4) Our replay
430 and grading run outside its loop. (5) The two models are reached through the scaffold's `litellm` layer,
431 GPT-5.6 through its `Responses-API` model class, which the `chat-completions` class cannot serve (it
432 returned three replies with no tool call and the scaffold exited on repeated format errors). (6) Prices
433 come from our verified table, because `litellm`'s carried a stale entry for one model. For these reasons
434 we claim no equivalence to its published resolve rate.

435 **Sampling grid.** Per run, median tree-changing observations and mutated files, with the run and final
436 observations excluded because neither is an opportunity to see a transition: public scaffold 3 and 3
437 (Claude), 2 and 2 (GPT-5.6); no recovery 2 and 2; repair-in-place 4 and 4; rollback 5 and 7; gate 4
438 and 4. Observations at which more than one file changed are 0 to 6% in every arm. The scaffold
439 issues 14.8 and 9.1 bash commands per observation against the harness's one tool call per observation,
440 because most of its commands only read.

441 **The self-test classifier.** For each episode we take the window in which the test was broken (the
442 breaking command up to, but not including, the command that repaired it) and label three things over
443 the scaffold's own trajectory. *Tested*: a test runner ran and its output proves it ran, so a runner that
444 died on an import does not count. *Covered*: one of those runs would have executed this test, decided
445 on the runner's positional arguments matched at a label boundary (a run of two sibling modules does
446 not cover a test in a third), honouring `-k`, `-m`, `--deselect`, and explicit node ids. *Shown*: a covering
447 run's output names this test on a line that also carries a failure marker. Coverage and display are
448 three-valued. A run that stopped at an earlier failure under `-x`, or whose output was truncated by a
449 pipe, is *unknown*, not negative, because calling it negative would assert that the agent ran the test and
450 was not told, from evidence that cannot support it. We audited the classifier's labels by hand against
451 the raw trajectories of eight bearing cells from an earlier pilot of this arm, run before the two shell
452 defects below were fixed and so not among the runs reported here. The audit disagreed with an earlier
453 version of the classifier on all three totals, and the version reported here reproduces the audit exactly.

454 **The gate's two contaminated Live trees.** One run hit the \$4 cap before the gate saw its final tree.
455 The other passed every check yet fails 13 grader tests, 7 in the file the hidden test patch rewrites.

456 **Uses.** The per-observation verdicts are the step-level ground truth that patch verifiers [24] and process
457 reward models [25] require. We release the archived timelines for that purpose.

458 **Further limitations.** Event counts miss in one direction and exaggerate in the other. A break that
459 stops the suite from being collected opens no timeline event, so the count understates what the agent
460 broke (Section 5.2). One edit can break many test functions at once, so it overstates how many
461 separate mistakes were made, which is why the undercount is reported in matched units and the
462 storms are named. Three runs supply 81% of the scaffold's 135 test functions: `seaborn-3069` under
463 both stacks (49 and 33) and `django-13121` (28). On the harness side `seaborn-3069` (73 declared events
464 over 39 test functions), `django-15128` (48) and `flask-5014` (41) supply 88% of the 184. Separately,
465 no-recovery's 19-of-19 persistence is confounded with its stopping rule, since that arm ends at the
466 monitor's first FAIL. To separate policy from truncation we would need a no-recovery arm that keeps

467 executing after a failed check. The public-scaffold arm is one scaffold at one version under two
468 model families. Its budget differs in kind from our arms' (\$3 billed and 250 steps against a \$4 work
469 cost), so the two harnesses are placed side by side and never tested against each other. The self-test
470 measurement rests on 11 breaking edits, one of them coded unknown. Cross-stack comparisons
471 confound model with adapter. The gated arms are post hoc and select tests from benchmark metadata.
472 Live resolve is near floor, and under rollback most Live runs ship nothing, so its recovery contrast is
473 uninformative on resolve.

474 **Bounding the model-stack difference (Section 5.2).** It does not replicate on Live (3 of 39 bearing
475 against 6, $p = 0.24$). Exposure differs about twofold, not 140-fold. In opportunity units the Claude
476 arm makes 147 observations to the GPT arm's 218, with per-run medians of 3 observations and 3
477 mutated files against 5 and 7. The zero is measured, not a detection failure. Re-scoring that stack's
478 earlier canary run reproduces its three events. A second plain-rollback sweep reproduced the GPT
479 side (13 of 36 graded resolved, 116 events, 6 of 37 bearing), so the bearing fraction is stable even
480 though per-instance event counts are not.

481 **The Claude rollback arm's contaminated tree.** django-14238, a run whose tree never changed.
482 Grading its unmodified base tree three times returns 38 of 39 previously-passing tests each time. One
483 graded id never appears in the log, which upstream's rule scores as a failure. It is baseline rot of the
484 kind Appendix A.3 describes, not a regression the timeline missed.

485 **The two readings.** The graded prediction excludes test files, as the official harness restores them
486 anyway, while the replay reads the tree as it stands, so an agent edit to a test file is visible to the
487 timeline and not to the grader.

488 **Reading Table 1.** *Baseline-dead* tests already fail in the unmodified image. *Rot-aware* resolve
489 counts a run resolved when every target test passes and no failure lies outside that set. On Live,
490 **resolved** is of 40 while **empty** and **bearing** are of the 39 attempted (torch-tune-1806 exceeds the
491 replay budget). **broken** $\rightarrow$ **left** omits any cell whose suite the grader could not collect, since such a
492 cell would otherwise read as perfect capture, so repair-in-place's row covers six of its seven bearing
493 runs and 83 of its 84 events.

494 **Incident exposure.** The pre-declared co-primary is a run's incident count, not an arm's total; Table 1's
495 **inc.** column gives the totals.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the scope (a pilot on a 40-instance slice per substrate, one sweep per arm), the five contributions, and that all measurements are exploratory; Section 6 lists the limitations that bound them.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 and Appendix C (Further limitations) cover slice size, single sweeps, the adapter confound of the cross-stack comparison, the post hoc gated arms and their selection leakage, the public-scaffold arm’s departures from that scaffold’s own configuration, Live resolve near floor, and event counts as bounds that miss in one direction and exaggerate in the other; Section 5.3 discloses the grading correction made after unblinding.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 3 specifies the instrument (commit per mutating call, replay of previously-passing tests in the pinned container, attribution, validation gates), Section 4 the substrates, slice rules, harness, tools, model identifiers, cost caps, outcomes and pre-declaration, and Appendix B the analysis plan; the defect catalogue (Appendix A) records the measurement pitfalls a replication must avoid.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The benchmark data are public (SWE-bench Verified, SWE-bench Live). The instrument, the sweep driver, the analysis scripts and the archived timelines will be released with the camera-ready version; they are not attached to the anonymous submission.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: No models are trained. Section 4 gives the slices and how they were fixed, the harness and its three recovery policies (the two regression-gated variants are specified in Section 5.4), the public scaffold and its unmodified budgets, the exact model strings, the per-run cost cap, the outcome definitions, and the pre-declared analysis.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The Claude rollback arm’s resolve carries an exact (Clopper–Pearson) 95% interval; paired comparisons use exact sign tests and exact McNemar tests, and the cross-stack comparison a one-sided Fisher exact test. The tests reported and the absence of multiplicity adjustment are declared (Section 5.4, Appendix B). Plain rollback was swept twice on Verified (Section 4, replicate in Appendix C), and intervals treat instances as independent although 16 of the 40 Verified instances come from django.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 3 gives replay cost per observation and per sweep; Table 1 gives model spend per arm and substrate; Appendix B states the machine (one 12-vCPU, 22 GB x86 virtual machine running Docker, no GPUs) and the total model spend including pilots.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn’t make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work measures software agents on public benchmarks; no human subjects, personal data, or high-risk assets are involved, and the submission is anonymized.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The contribution is an evaluation instrument and measurements of it, and the plausible direct impact is positive: agents whose regressions are visible break less existing behaviour, and the archived timelines give patch verifiers and process reward models step-level supervision they currently have to guess at. Two negative paths are worth naming. An instrument that commits the working tree after every action is agent telemetry: deployed outside a benchmark it would record whatever an agent touches, including proprietary or personal code, so the hidden reference is a surveillance surface as much as a measurement one. And a gate that rejects any step failing a prior-passing test is an enforcement mechanism, not only an observer: applied to real repositories it would suppress legitimate refactors that temporarily break tests, which is the behaviour a maintainer sometimes wants. Both argue for the gate being advisory and the timeline being local and short-lived by default.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No models or scraped datasets are released.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: SWE-bench, SWE-bench Verified and SWE-bench Live are cited [1, 2, 3] and used under their MIT licenses and published container images; the models are accessed through their providers' APIs under the providers' terms; every cited artefact appears in the references.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [No]

Justification: The instrument and the archived timelines are not released with the anonymous submission; they will be released, documented, with the camera-ready version.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing or human-subject research.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are the agents under measurement: the planner, worker and monitor models are named by their API identifiers in Section 4, and their roles in the harness are described in Sections 3 and 4.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.